

TP previo Parcial 2 DDS 2026 - Curso 3K6

Campo	Valor
Materia	Desarrollo de Software (DDS)
Año	2026
Instancia de evaluación	TP previo - Parcial 2
Documento	Enunciado de trabajo práctico
Curso	3K6

Introducción

Durante una semana el grupo va a desarrollar una aplicación full stack para control de mantenimiento. Desde el 14/6 10 hs hasta el lunes 15/6 a las 23:55 otro grupo deberá realizar una revisión/corrección entre pares (revisión entre grupos) y entregar la devolución grupal.

Adicionalmente cada alumno deberá subir el trabajo realizado a su portafolio.

Estas dos tareas son esenciales y previas al segundo parcial ya que habrá 2 preguntas a desarrollar relativas al tema desarrollado.

El sistema debe registrar órdenes sobre activos, asignar técnicos, priorizar incidencias, controlar estados y reflejar impacto sobre los activos. El objetivo es construir una herramienta operativa con decisiones de dominio, trazabilidad y permisos, no solo un formulario de reclamos.

Este práctico integra los contenidos principales del segundo parcial: Express Router, middlewares, manejo centralizado de errores, CORS, autenticación y autorización con JWT, pruebas de API con Jest y Supertest, React con Vite, componentes, estado, efectos, React Router, formularios, Axios, validaciones y manejo de errores de API.

Como el trabajo se resuelve en grupo y con una semana de desarrollo, se espera una solución de complejidad media-alta: reglas de negocio reales, estados, permisos, historial, filtros avanzados, persistencia, datos semilla, pruebas automatizadas y una interfaz navegable que permita operar el sistema completo.

Alcance mínimo de resolución grupal

La entrega debe evidenciar que hubo división de responsabilidades y posterior integración. Como mínimo, el sistema debe incluir:

- un módulo de autenticación completo, con registro, login, JWT, roles y persistencia de sesión en **frontend**;
- un módulo principal del dominio con listado, filtros, detalle, alta, edición y acciones de estado;
- un módulo de administración o resumen que muestre información agregada del dominio;
- validaciones de negocio implementadas en servicios del backend, no solamente en formularios;
- al menos dos flujos end-to-end funcionando desde la interfaz hasta la persistencia;

- pruebas automatizadas que cubran casos exitosos, errores de validación, errores de permisos y **reglas específicas del dominio**;
- documentación suficiente para que otro grupo pueda ejecutar, probar y explicar la solución.

No se considera suficiente una entrega que solo tenga pantallas sin backend real, endpoints sin frontend integrado, datos hardcodeados en componentes, o reglas de negocio aplicadas únicamente de forma visual.

Que tienen que construir?

Desarrollar una aplicación web full stack para control de órdenes de mantenimiento. La aplicación debe permitir registrar órdenes sobre activos, priorizarlas, asignarlas y controlar su estado. No es un

CRUD genérico: una orden debe estar asociada a un activo valido y sus cambios deben respetar prioridad, estado y permisos.

En el parcial presencial cada estudiante respondera preguntas escritas sobre la solución entregada por su grupo. Deben poder explicar rutas, componentes, servicios, middlewares, validaciones, JWT, permisos, pruebas y errores manejados.

Dominio obligatorio

El sistema trabaja sobre el dominio de control de mantenimiento. El recurso principal que se administra es la Orden: una solicitud de intervencion sobre un activo, con prioridad, técnico asignado, estado y fechas de seguimiento. Cada orden debe estar asociada obligatoriamente a un Activo, que representa el elemento que requiere mantenimiento.

La regla central del dominio es que una orden solo puede crearse si el activo existe y no está dado de baja. Además, la prioridad, el estado y la asignación de técnico deben ser coherentes con la criticidad del activo y con el avance real del mantenimiento.

Entidades mínimas obligatorias

El sistema debe modelar y persistir al menos 4 entidades. Sí usan archivo JSON, SQLite, Sequelize u otra alternativa, igual deben respetar está separacion lógica:

Entidad	Para qué existe	Campos mínimos	Relaciones principales
usuarios	Representa solicitantes, técnicos y administradores de mantenimiento.	id, nombre, email, passwordHash, rol, activo	Un usuario puede crear órdenes, ser técnico asignado y generar historial.
activos	Representa elementos que pueden requerir mantenimiento.	id, codigo, nombre, tipo, ubicacion, estado, criticidad	Un activo puede tener muchas órdenes.
órdenes	Representa una intervención solicitada sobre un activo.	id, activoId, titulo, descripcion, solicitanteId, tecnicoId, prioridad, estado, fechaCreacion, fechaResolución	Pertenece a un activo, tiene solicitante y puede tener técnico asignado.
historial_órdenes	Registra auditoría de cambios sobre órdenes.	id, ordenId, usuarioId, accion, fechaHora, valorAnterior, valorNuevo	Pertenece a una orden y al usuario que hizo la acción.

Campos mínimos por entidad

Los campos pueden adaptarse al motor de persistencia elegido, pero la información y las relaciones deben estar presentes.

Usuario

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único del usuario.	1, usr-001
nombre	string	Sí	Nombre visible del usuario.	Pablo Díaz

email	string	Sí	Credencial de acceso y dato de contacto. Debe ser único.	pablo@dds.com
passwordHash	string	Sí	Contraseña almacenada de forma segura o hash simulado documentado.	bcrypt_hash
rol	string	Sí	Define permisos dentro del sistema.	solicitante, técnico, mantenimiento, admin
activo	boolean	Sí	Permite bloquear accesos sin borrar el usuario.	true, false

Activo

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único del activo.	act-001
codigo	string	Sí	Código interno o patrimonial. Debe ser único.	AIRE-2P-03
nombre	string	Sí	Nombre reconocible del activo.	Aire acondicionado aula 3
tipo	string	Sí	Clasificación del activo.	equipo, instalación, mobiliario, software
ubicacion	string	Sí	Lugar donde se encuentra.	Ed. Possetto - Aula 3
estado	string	Sí	Estado operativo actual.	operativo, con_falla, en_mantenimiento, baja
criticidad	string	Sí	Impacto del activo sobre la operación.	baja, media, alta

Orden

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único de la orden.	ord-1001
activoId	número/string	Sí	Activo afectado. Debe existir en activos.	act-001
titulo	string	Sí	Resumen corto del problema.	No enfría

descripcion	string	Sí	Detalle del incidente o mantenimiento requerido.	El equipo enciende pero no baja temperatura
solicitanteId	número/string	Sí	Usuario que crea la orden.	usr-001
tecnicoId	número/string	No	Usuario técnico asignado. Obligatorio para resolver.	usr-tec-1
prioridad	string	Sí	Urgencia de atención.	baja, media, alta, urgente
estado	string	Sí	Estado actual del flujo.	abierta, asignada, en_proceso, resuelta, cancelada
fechaCreacion	datetime/string	Sí	Fecha y hora de creación.	2026-06-08T10:30:00
fechaResolución	datetime/string	No	Fecha y hora de resolución.	2026-06-10T16:00:00

HistorialOrden

Campo	Tipo sugerido	Obligatorio	Descripción	Ejemplo o valores posibles
id	número/string	Sí	Identificador único del registro de auditoría.	hist-001
ordenId	número/string	Sí	Orden afectada por el cambio.	ord-1001
usuarioId	número/string	Sí	Usuario que realizó la acción.	usr-admin
accion	string	Sí	Operación realizada sobre la orden.	creacion, asignación, cambio_prioridad, cambio_estado, resolucion, cancelación
fechaHora	datetime/string	Sí	Momento exacto de la acción.	2026-06-08T11:15:00
valorAnterior	object/string	No	Datos relevantes antes del cambio.	{ "estado": "abierta" }
valorNuevo	object/string	No	Datos relevantes después del cambio.	{ "estado": "asignada" }

Reglas funcionales específicas

La aplicación debe resolver estos casos:

1. Registrar usuario e iniciar sesión.
2. Obtener un JWT al iniciar sesión correctamente.
3. Listar órdenes con filtros combinables por activoId, estado, prioridad y técnicoId.
4. Ver el detalle de una orden desde una ruta dinámica.
5. Crear una orden seleccionando un activo existente.
6. Editar titulo, descripcion, prioridad, técnico o estado según permisos.
7. Cancelar una orden cambiando estado a cancelada.
8. Rechazar órdenes para activos inexistentes o en estado baja.
9. Rechazar prioridades y estados fuera de los valores permitidos.
10. Rechazar marcar como resuelta una orden que no tenga técnicoId asignado.

Complejidad adicional obligatoria

El grupo debe implementar también:

1. Flujo completo de estados: abierta → asignada; asignada → en_proceso; en_proceso → resuelta; cualquier orden no resuelta → cancelada. No permitir resolver una orden cancelada.
2. Búsqueda paginada y ordenable: aceptar page, limit, sortBy y order en el listado de órdenes.
3. Vista resumen para administración con órdenes por estado, órdenes urgentes, activos con mas fallas y órdenes sin técnico.
4. Historial de cambios de cada orden: guardar fecha, usuario, acción y valores modificados cuando se asigna, cambia prioridad, cambia estado o se edita.
5. Calculo de prioridad sugerida: si el activo tiene criticidad alta, la orden no puede quedar con prioridad baja.
6. Cambio controlado del estado del activo cuando se crea una orden sobre activo operativo, cuando se asigna mantenimiento y cuando se resuelve.
7. Semilla de datos inicial con al menos 8 activos, 3 técnicos, 1 admin/mantenimiento y 15 órdenes en distintos estados.
8. Mensajes de error distintos para activo inexistente, activo dado de baja, prioridad inválida, técnico requerido y falta de permisos.

Aclaraciones para resolver el alcance

Estas aclaraciones responden dudas esperables al momento de implementar:

- Sobre que activos se pueden crear órdenes? Solo sobre activos existentes que no esten en estado baja.
- Como se relaciona criticidad con prioridad? Sí el activo tiene criticidad alta, la orden no puede

quedar con prioridad baja.

- Cuando se puede resolver una orden? Solo si tiene técnico asignado y está en un estado compatible con el flujo, por ejemplo en_proceso.
- Que pasa con el estado del activo? Al abrir o asignar una orden puede pasar a con_falla o en_mantenimiento; al resolverla puede volver a operativo, según criterio documentado.
- Que debe mostrar el resumen? Como minimo: órdenes por estado, órdenes urgentes, órdenes sin técnico y activos con mas fallas.
- Donde se validan prioridad, técnico y activo? En el servicio del backend. El frontend no debe ser la unica barrera.

Roles y permisos

Deben existir al menos estos roles:

solicitante: puede crear órdenes, ver las propias y cancelar órdenes abiertas propias.

técnico: puede ver órdenes asignadas y pasar sus órdenes a en_proceso o resuelta.
admin o mantenimiento: puede ver todas las órdenes, asignar técnico, cambiar prioridad y cancelar cualquier orden.

Al menos una ruta de escritura debe devolver:

401 si no se envia JWT.

403 si el usuario está autenticado pero no tiene permiso para la acción.

Backend esperado

Implementar backend con Node.js y Express.

Rutas mínimas:

POST /api/auth/register

POST /api/auth/login

GET /api/activos

GET /api/órdenes?activoId=&estado=&prioridad=&técnicoId=

GET /api/órdenes/resumen

GET /api/órdenes/:id

GET /api/órdenes/:id/historial

POST /api/órdenes

PUT /api/órdenes/:id

PATCH /api/órdenes/:id/cancelar

PATCH /api/órdenes/:id/asignar

PATCH /api/órdenes/:id/resolver

Estructura minima:

- `routes/órdenes.routes.js` con `express.Router()`
- controlador de órdenes
- servicio de órdenes con reglas de activo, prioridad y estado
- middleware de autenticación JWT
- middleware de autorización por rol, solicitante o técnico asignado
- middleware de validación de entrada
- middleware de manejo de errores con firma (`err, req, res, next`)
- persistencia en archivo JSON, SQLite, Sequelize u otra solución que conserve datos al reiniciar el **backend**
- carga de datos semilla para probar el dominio sin cargar todo manualmente

Usar status HTTP coherentes: 200, 201, 400, 401, 403, 404 y 500. Las respuestas de error deben tener JSON claro, por ejemplo { "error": "No se puede crear una orden sobre un

activo dado de baja" }.

Frontend esperado

Implementar frontend con React y Vite.

Pantallas mínimas:

1. Login y registro.
2. Listado de órdenes con filtros por activo, estado, prioridad y técnico.
3. Detalle de orden en una ruta como /órdenes/:id.
4. Pantalla transaccional de alta/edición de orden, donde se selecciona activo, prioridad, técnico cuando corresponda y descripción, y se confirma la operación contra la API.
5. Acciones visibles para asignar, cancelar o resolver según rol.
6. Panel resumen para administradores o mantenimiento.
7. Historial visible dentro del detalle de una orden.
8. Ruta comodín para página no encontrada.

La capa de servicios debe usar Axios:

1. **instancia con baseUrl**
2. **params para filtros**
3. **body en POST, PUT o PATCH**
4. **header Authorization: Bearer <token> en acciones protegidas**
5. **manejo visible de errores de validación, autenticación, autorización y recurso inexistente**
6. **componentes separados para tabla/listado, filtros, formulario, detalle, acciones por rol y resumen administrativo**
7. **estados de carga, vacío, error y éxito para las operaciones principales**

Integracion y calidad esperada

La resolución debe mostrar trabajo de equipo y no una acumulacion de archivos sueltos. Se espera:

- ✓ contexto, hook o mecanismo equivalente para conservar usuario autenticado, token y rol en **frontend**;
- ✓ **rutas protegidas en frontend y backend, no solo botones ocultos**;
- ✓ validaciones repetidas en frontend para experiencia de usuario y en backend como fuente de **verdad**;
- ✓ servicios Axios separados por recurso, sin llamadas HTTP mezcladas dentro de todos los **componentes**;
- ✓ contraseñas hasheadas o, si usan usuarios semilla simplificados, aclaracion explicita de la **limitacion en el README**;

- ✓ **payload del JWT sin contraseñas ni datos sensibles;**
- ✓ **manejo de errores centralizado en backend y mensajes comprensibles en frontend;**
- ✓ **decision documentada sobre persistencia, estructura de carpetas y división de responsabilidades del grupo.**

Testing minimo

El backend debe incluir pruebas con Jest y Supertest para:

1. Login correcto e inválido.
2. Listado de órdenes con y sin filtros.
3. Detalle de orden existente e inexistente.
4. Creacion valida de una orden.
5. Creacion inválida por activo dado de baja.
6. Resolución inválida de orden sin técnico asignado.
7. Acceso sin JWT a una ruta protegida.
8. Acceso con JWT de solicitante a una acción solo permitida para admin o mantenimiento.
9. Creacion inválida con prioridad baja para activo de criticidad alta.
10. Transicion de estado no permitida, por ejemplo resolver una orden cancelada.

Las pruebas deben verificar status HTTP y cuerpo JSON relevante.

Documentacion obligatoria

La entrega debe incluir un README.md con:

- ✓ **instrucciones para ejecutar backend y frontend**
- ✓ **usuario admin o mantenimiento, usuario técnico y usuario solicitante de prueba**

- ✓ listado de endpoints principales
- ✓ listado de rutas frontend
- ✓ explicación de validación de activo, prioridad y estado
- ✓ explicación de JWT, roles y permisos
- ✓ comando para ejecutar pruebas
- ✓ limitaciones conocidas

Preguntas frecuentes y criterios de correccion

Sobre alcance y evaluacion

El TP es condicion de entrega y también sera la base de preguntas abiertas del Parcial 2. La nota del parcial surge del examen, pero una entrega incompleta o no explicable afecta directamente

las respuestas escritas.

En el parcial se preguntara sobre el código real entregado por el grupo: rutas, servicios, componentes, validaciones, JWT, permisos, pruebas y decisiones.

Se aceptan soluciones equivalentes si respetan entidades mínimas, reglas del dominio, rutas, pantallas, pruebas y criterios de seguridad. Cualquier cambio importante debe estar justificado en el README.

La persistencia puede ser archivo JSON, SQLite, Sequelize u otra alternativa, siempre que conserve datos al reiniciar y respete las entidades del enunciado.

Debe haber registro/login real desde frontend. Los usuarios semilla son obligatorios para probar rapido, pero no reemplazan el flujo de autenticación.

El historial debe existir en backend y verse desde frontend, al menos en el detalle del recurso principal.

La pantalla transaccional puede ser una sola pantalla para alta/edición o dos pantallas separadas, pero debe confirmar la operación contra la API y mostrar errores.

El resumen administrativo debe estar protegido para roles administrativos.

Filtros, paginación y ordenamiento deben resolverse en backend. El frontend solo envia params y muestra resultados.

Documentar una limitacion no reemplaza un requisito minimo. Sirve para explicar decisiones o faltantes menores, no para omitir reglas centrales.

Sobre backend

Deben usar `express.Router()` en archivos separados para el recurso principal.

El middleware de errores con firma (`err, req, res, next`) debe quedar después de las rutas.

Los errores de validación deben responder con status coherente, normalmente 400, y JSON claro, por ejemplo `{ "error": "mensaje" }`.

La autorización debe validar rol y, cuando corresponda, propiedad del recurso. No alcanza con que el JWT exista.

El JWT no debe incluir contraseñas ni datos sensibles.

Las contraseñas deben guardarse hasheadas; si se usa una simplificacion por tiempo, debe estar documentada y no exponer contraseñas en respuestas.

Las reglas de negocio deben estar en servicios del backend, no solamente en controladores ni

formularios.

Deben estar protegidas las acciones de escritura y cualquier lectura privada o administrativa.

Sobre frontend

React Router debe incluir ruta comodin *.

Las pantallas de detalle deben leer parametros con useParams.

Pueden usar React Hook Form o formularios controlados con useState, pero las validaciones deben verse en pantalla.

Los errores de API deben mostrarse de forma comprensible cerca de la acción que fallo o en un area visible de la pantalla.

Axios debe estar en una capa de servicios separada. No mezclar llamadas HTTP en todos los componentes.

El JWT puede enviarse con interceptores o agregando headers en servicios, pero siempre como Authorization: Bearer <token>.

Proteger una ruta en frontend significa impedir acceso visual/navegable si no hay usuario autenticado o rol suficiente; esto no reemplaza la proteccion del backend.

Sobre testing

Se exigen pruebas de backend con Jest y Supertest. Tests de frontend son opcionales.

Cada test debe validar status HTTP y cuerpo JSON relevante.

Deben probar login correcto, login inválido, acceso sin token y acceso con rol insuficiente.

Deben probar reglas específicas del dominio, no solo endpoints felices.

Los datos de prueba deben ser previsibles. Sí un test modifica datos, deben resetearlos o aislarlos para no afectar otros tests.

Entrega

Subir a Moodle el repositorio o archivo comprimido en el portafolio del alumno y en la entrega grupal. No incluir carpetas node_modules. Sí deben incluir package.json, código fuente, tests y documentación.